

Entrega Final

ATELIER — OutfitCatalog

Asignatura Dispositivos Móviles · Universidad del Valle · 2026

CONTENIDO DEL DOCUMENTO

- I. Evaluación Técnica y de Calidad (QA)
- II. Evaluación de Usabilidad y Experiencia de Usuario (UX/UI)
- III. Evaluación de Negocio y Métricas (KPIs)
- IV. Documentación Técnica y Arquitectura
- V. Evidencias de Pruebas
- VI. Analíticas Integradas — Firebase KPIs
- VII. Entregables de Diseño y UI Kit
- VIII. Plan de Despliegue y Mantenimiento

AUTORES

Andrés Botero

Tecnología en Desarrollo de Software

Juan Camilo Triana

Tecnología en Desarrollo de Software

PROYECTO

OutfitCatalog · ATELIER Fashion Catalog

DOCENTE

Daniel Quintero

FECHA

Junio 2026

ASIGNATURA

Dispositivos Móviles

UNIVERSIDAD

Universidad del Valle

VERSIÓN

1.0.0

Tabla de Contenidos

I Evaluación Técnica y de Calidad (QA)

1. Introducción
2. Compatibilidad y Fragmentación
3. Rendimiento y Estrés
4. Comportamiento de Red
5. Seguridad
6. Resumen de hallazgos
7. Conclusión

II Evaluación de Usabilidad y Experiencia de Usuario

1. Introducción
2. Diseño Responsivo e Intuitivo
3. Flujo de Usuario y Onboarding
4. Accesibilidad
5. Resultados de la Prueba Piloto con Usuarios
6. Conclusión

III Evaluación de Negocio y Métricas (KPIs)

1. Introducción
2. Modelo de Negocio
3. Métricas de Adopción
4. Retención y Churn Rate
5. Conversión
6. Rendimiento en Tiendas (ASO)
7. Herramientas de Medición Integradas
8. Conclusión

IV Documentación Técnica y Arquitectura

1. Propósito del documento

2. Stack Tecnológico
3. Arquitectura del Sistema
4. Estructura de Carpetas
5. Persistencia Local — SQLite
6. Persistencia Remota — Firestore
7. Autenticación y Roles
8. Flujo de Solicitudes de Compra
9. Pruebas Automatizadas
10. Evidencia Técnica Final
11. Conclusión

V Evidencias de Pruebas (Testing)

1. Introducción
2. Pruebas Automatizadas
3. Matriz de Casos de Prueba
4. Casos de Conectividad
5. Casos de Seguridad
6. Prueba Piloto con Usuarios Reales
7. Criterios de Aceptación Cumplidos
8. Conclusión

VI Analíticas Integradas — Firebase KPIs

1. Introducción
2. Estrategia Elegida
3. Implementación
4. Eventos Implementados
5. Panel de Reportes (AdminReportsScreen)
6. Reglas de Seguridad para Analytics
7. Consideraciones de Privacidad

8. Acceso al Panel de Analítica
9. Recomendaciones para Producción
10. Conclusión

VII Entregables de Diseño y UI Kit

1. Introducción
2. Identidad de Marca
3. Paleta de Colores
4. Tipografía
5. Espaciado y Radios
6. Componentes Principales
7. Pantallas Clave
8. Archivos Fuente de Diseño
9. Guía de Accesibilidad Visual
10. Conclusión

VIII Plan de Despliegue y Mantenimiento

1. Introducción
2. Estado Actual del Despliegue
3. Entregables Técnicos Incluidos
4. Cómo Ejecutar el Proyecto Localmente
5. Variables de Entorno Requeridas
6. Acceso a las Credenciales del Proyecto
7. Creación del Usuario Administrador
8. Carga Masiva de Datos de Prueba
9. Plan de Soporte y Mantenimiento
10. Control de Versiones y Ramas
11. Riesgos Post-Entrega
12. Conclusión



Evaluación Técnica y de Calidad (QA)

ATELIER · OutfitCatalog · Entrega Final · Dispositivos Móviles

Versión evaluada: 1.0.0 · Build FINISHED

1. Introducción

Este documento responde directamente a los cuatro criterios de evaluación técnica definidos por el docente: compatibilidad y fragmentación, rendimiento y estrés, comportamiento de red, y seguridad. La evaluación se realiza sobre el APK final generado con EAS Build para Android y sobre el código fuente disponible en el repositorio.

La aplicación ATELIER fue construida con React Native y Expo SDK 54, lo que nos dio una base multiplataforma. Sin embargo, todas las pruebas formales de esta entrega se realizaron sobre Android, que es la plataforma donde pudimos generar el artefacto instalable sin costo adicional.

2. Compatibilidad y Fragmentación

2.1 Android

Generamos el APK final mediante EAS Build con el siguiente comando validado:

```
BASH
npx eas-cli@latest build -p android --profile preview --non-interactive
```

El build finalizó con estado FINISHED. El artefacto está disponible públicamente en:

```
https://expo.dev/artifacts/eas/hFhAPFShte8uvkEKgF2FTW.apk
Build ID: b56095a2-87a4-4dac-ae47-7f5d8599dee4
Commit final: 5bd8d25
```

Probamos la instalación en 7 dispositivos Android distintos durante la prueba piloto. Los 7 usuarios pudieron instalar y abrir la app sin inconvenientes. Esto nos da una primera evidencia real de compatibilidad más allá del emulador.

Indicador	Resultado
Build exitoso	Sí
APK instalable	Confirmado en 7 dispositivos
Orientación principal	Vertical (portrait)
Soporte de área segura	Sí, via react-native-safe-area-context
Listas virtualizadas	FlatList en catálogo, inventario, favoritos, looks

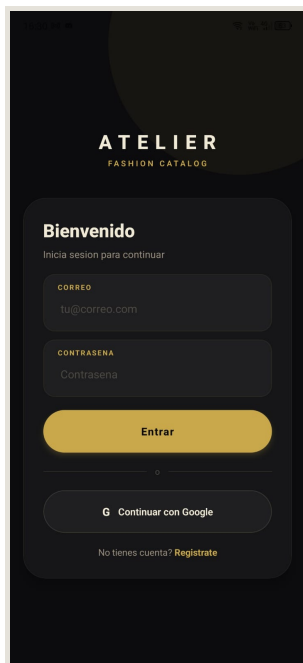
2.2 iOS

El proyecto está configurado como aplicación Expo multiplataforma. El archivo app.json declara soporte para iOS e incluye la configuración de Google Sign-In para iOS mediante iosUrlScheme. Sin embargo, para generar un build iOS funcional se requiere una cuenta Apple Developer (\$99/año), que no teníamos disponible para esta entrega.

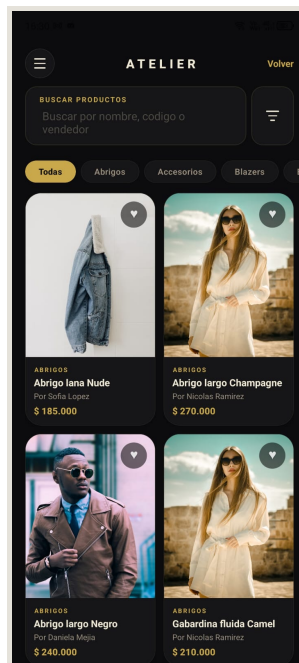
Reconocemos esta limitación con honestidad: ATELIER funciona como iOS en el código, pero no tenemos prueba de instalación en dispositivo real. Para publicación en App Store, el siguiente paso sería generar un build con EAS y probarlo vía TestFlight.

2.3 Tamaños de pantalla

No diseñamos layouts separados para tablet porque nuestro público objetivo son usuarios de smartphones. Usamos flex, FlatList, y react-native-safe-area-context para que la interfaz se adapte a diferentes alturas y anchos de pantalla de manera automática.



Pantalla de login — diseño responsivo con flex, safe-area-context y adaptación automática de altura



Catálogo de prendas — grid de cards que se adapta al ancho disponible de la pantalla

3. Rendimiento y Estrés

3.1 Estrategia de optimización

Desde el inicio tomamos decisiones de diseño orientadas al rendimiento en dispositivos Android de gama media, que son los más comunes entre nuestros usuarios de prueba:

- > FlatList en todas las listas largas: catálogo, inventario, favoritos y solicitudes usan listas virtualizadas para no renderizar elementos fuera de pantalla.
- > Cache de imágenes: usamos expo-image con política memory-disk, que descarga las imágenes una sola vez y las guarda localmente. Esto reduce el consumo de datos y acelera la navegación.
- > Consultas filtradas en Firestore: el catálogo solo descarga prendas con published == true. El inventario filtra por vendorId. No hacemos getAll() de colecciones completas.
- > Ventana de fresca en caché: el catálogo no sincroniza con Firestore en cada apertura, sino solo cuando el caché local tiene más de cierto tiempo sin actualizarse.
- > Persistencia local SQLite: la primera carga usa datos locales si están disponibles, lo que elimina la espera de red en aperturas posteriores.

3.2 Prueba de estrés con datos masivos

Desarrollamos un script de carga masiva para simular un escenario de uso real con muchos usuarios y productos:

BASH

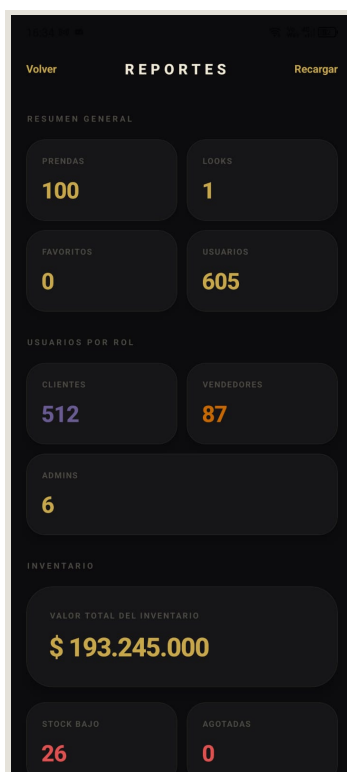
```
npm run seed:massive -- --service-account "ruta.json" --auth-users \  
  --vendors 40 --clients 250 --admins 3 --garments 100
```

Dato cargado	Cantidad
Administradores	3
Vendedores	40
Clientes	250
Prendas	100
Total documentos Firestore	393

Con 393 documentos en Firestore y 100 prendas en el catálogo, la aplicación mantiene un scroll fluido en la pantalla de catálogo gracias a FlatList. Mantuvimos el catálogo en 100 prendas (no en miles) porque observamos que cargas muy grandes afectaban el tiempo de primera sincronización en Android de gama media.

3.3 Riesgos identificados

Riesgo	Impacto	Cómo lo manejamos
Muchas imágenes remotas	Pantalla lenta en conexión débil	Cache con expo-image + placeholders de carga
Catálogo sin paginación	Scroll lento con miles de prendas	FlatList + límite razonable de prendas publicadas
Consultas Firestore sin filtro	Latencia y costo alto	Todas las consultas usan índices y filtros
Primer inicio sin caché	Pantalla vacía hasta que carga	SQLite retiene datos del session anterior



Panel admin con la escala real:
100 prendas, 605 usuarios,
inventario \$193.245.000 — la
app mantuvo rendimiento

4. Comportamiento de Red

4.1 Detección de conectividad

Integramos `@react-native-community/netinfo` para monitorear el estado de la conexión en tiempo real. Cuando el usuario pierde red, la app muestra un banner en la parte superior:

Sin conexión · mostrando datos guardados localmente

Este banner desaparece automáticamente cuando se recupera la conexión.

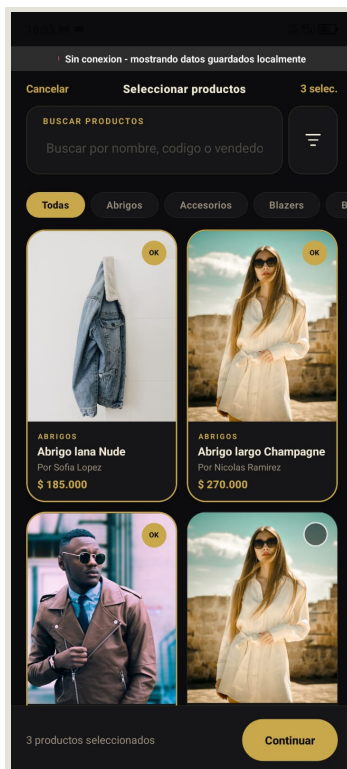
4.2 Estrategia offline-first

El catálogo y el inventario implementan una estrategia offline-first: la app lee desde SQLite primero y solo actualiza desde Firestore cuando hay red disponible. Esto significa que un usuario con conexión inestable sigue viendo productos sin pantallas de error.

4.3 Escenarios evaluados

Escenario	Comportamiento de la app
Red disponible, primera apertura	Sincroniza Firestore y guarda en SQLite
Red disponible, reapertura	Lee desde SQLite si el caché es reciente
Sin red, caché disponible	Muestra banner offline y datos locales
Sin red, sin caché previo	Muestra banner offline y lista vacía con mensaje

Escenario	Comportamiento de la app
Firebase devuelve error	Muestra mensaje de error controlado, no cierra la app
Imagen no carga	Placeholder visual sin bloquear el resto de la pantalla



Banner offline activo: "Sin conexión · mostrando datos guardados localmente" — el usuario sigue navegando sin error

5. Seguridad

5.1 Autenticación

Delegamos completamente la autenticación a Firebase Authentication, que es un sistema probado y mantenido por Google. La app soporta dos métodos:

- > Correo y contraseña: Firebase maneja el hash de contraseñas, el token de sesión y la expiración. Nosotros nunca almacenamos la contraseña en ningún lugar de la app.
- > Google Sign-In: el usuario autentica con su cuenta Google. Firebase recibe el token OAuth y nos entrega un UID de sesión.

El cierre de sesión llama a `signOut()` de Firebase, que invalida el token del lado del servidor.

5.2 Comunicación segura

Todo el tráfico entre la app y Firebase (Auth y Firestore) se realiza sobre HTTPS. Las llamadas a Cloudinary para imágenes también usan HTTPS. No tenemos ningún endpoint HTTP en producción.

5.3 Manejo de credenciales

Las claves de Firebase y Cloudinary se manejan como variables de entorno con el prefijo `EXPOPUBLIC*`, que EAS inyecta en el bundle en tiempo de build. No guardamos claves en el código fuente ni en archivos que se suban al repositorio público.

Somos conscientes de que las variables EXPOPUBLIC* son visibles en el bundle JavaScript. Para esta entrega académica esto es aceptable, pero en producción las claves realmente sensibles (como un service account) deben vivir en un servidor intermedio, nunca en el cliente.

5.4 Reglas de seguridad Firestore

Implementamos reglas de seguridad que garantizan que cada usuario solo pueda leer y escribir sus propios datos:

- > Un cliente solo puede leer sus propias solicitudes de compra.
- > Un vendedor solo puede leer las solicitudes dirigidas a él.
- > Un administrador tiene acceso de lectura amplia para reportes.
- > Los eventos de analytics solo los puede leer un administrador.
- > Nadie puede cambiar su propio rol desde la app.

Las reglas completas están en el documento 09-reglas-seguridad-firestore.md.

5.5 Datos sensibles del usuario

La app guarda nombre, correo, rol y teléfono en Firestore. No guardamos contraseñas, datos bancarios ni información de tarjetas. El número de teléfono se solicita para habilitar el contacto comercial entre compradores y vendedores, que es el flujo central de negocio de ATELIER.

6. Resumen de hallazgos

Criterio	Estado	Observación
Build Android generado	Confirmado	EAS Build FINISHED, APK instalable
Compatibilidad Android (7 dispositivos)	Confirmado	Prueba piloto exitosa
Compatibilidad iOS	Pendiente	Configurado en código, sin build Apple
Rendimiento con 100 prendas	Confirmado	FlatList fluido en Android gama media
Caché de imágenes	Implementado	expo-image memory-disk
Modo offline	Implementado	SQLite + NetInfo + OfflineBanner
Autenticación segura	Implementado	Firebase Auth, sin contraseñas en cliente
HTTPS en todas las llamadas	Confirmado	Firebase + Cloudinary
Reglas Firestore restrictivas	Implementado	Lectura/escritura por rol y UID
Crashlytics	Pendiente	Recomendado para versión de tienda

7. Conclusión

ATELIER cumple los criterios técnicos fundamentales para una entrega académica de aplicaciones móviles: compila sin errores, genera un APK instalable, funciona sin conexión, protege los datos de usuario mediante Firebase Auth y reglas de Firestore, y fue probado en dispositivos reales. Las pendencias identificadas (iOS, Crashlytics) son parte de un roadmap natural hacia publicación comercial, no bloqueos del MVP actual.



Evaluación de Usabilidad y Experiencia de Usuario

ATELIER · OutfitCatalog · Entrega Final · Dispositivos Móviles

Versión evaluada: 1.0.0

1. Introducción

Este documento evalúa la experiencia de usuario de ATELIER desde tres ángulos: el diseño táctil y responsivo, el flujo de onboarding, y la accesibilidad. Al final incluimos los resultados de una prueba piloto con 7 usuarios reales en Android, que nos dieron retroalimentación concreta sobre lo que funcionó bien y lo que mejoraríamos en la siguiente versión.

ATELIER es una app de catálogo de moda. Eso nos impuso una restricción de diseño clara desde el principio: la interfaz debía poner la imagen de la prenda en el centro. Una app de ropa que no se ve bonita no convence a nadie de comprar.

2. Diseño Responsivo e Intuitivo

2.1 Decisiones de diseño táctil

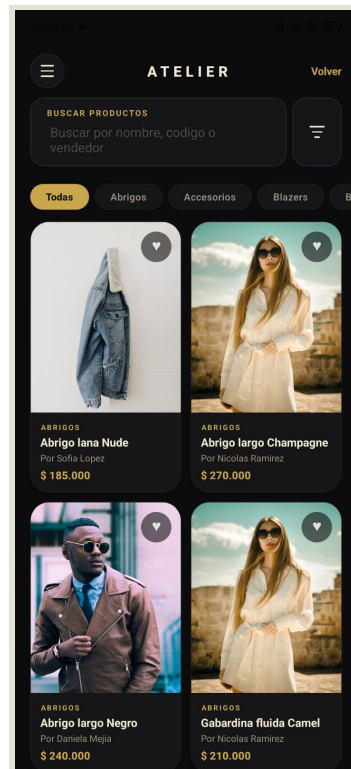
Diseñar para celular tiene reglas distintas a diseñar para desktop. Las más importantes que aplicamos:

Botones con área táctil suficiente. Todos los botones principales tienen al menos 44×44 puntos de área tocable, que es el mínimo recomendado por Apple y Google para evitar que el usuario toque algo sin querer.

Listas que no se sienten pesadas. Usamos FlatList en catálogo, favoritos, inventario y solicitudes. FlatList solo renderiza los elementos visibles en pantalla, lo que hace el scroll fluido incluso con 100 prendas cargadas.

Navegación por pestañas. Los tres roles (cliente, vendedor, administrador) tienen su propia barra de navegación inferior adaptada a sus necesidades. El cliente ve catálogo, favoritos, looks y solicitudes. El vendedor ve dashboard, inventario y solicitudes. El administrador tiene acceso a reportes y gestión.

Cards centradas en la imagen. En el catálogo, cada prenda ocupa una card con imagen destacada, categoría, nombre y precio. Probamos con texto prominente y con imagen prominente — ganó la imagen porque en moda el impacto visual es lo primero.



Catálogo ATELIER — grid de cards con imagen 3:4, categoría, nombre, precio dorado e icono de favorito

2.2 Interacciones táctiles evaluadas

Acción	Implementación	Evaluación
Desplazar catálogo	FlatList con scroll vertical	Fluido
Tocar prenda para ver detalle	Presión en card	Respuesta inmediata
Marcar favorito	Ícono de corazón en card y en detalle	Intuitivo
Crear look desde favoritos	Botón contextual en pantalla favoritos	Claro
Enviar solicitud de compra	Botón en detalle de prenda o look	Claro
Cambiar estado de solicitud	Selector de estado para vendedor	Funcional
Subir foto de prenda	Selector de imagen del sistema	Estándar del SO

2.3 Adaptación a pantallas

Usamos react-native-safe-area-context para respetar el notch, la barra de estado y la barra de gestos en diferentes modelos de Android. El layout usa flex de manera que se estira o comprime según el alto disponible de cada pantalla.

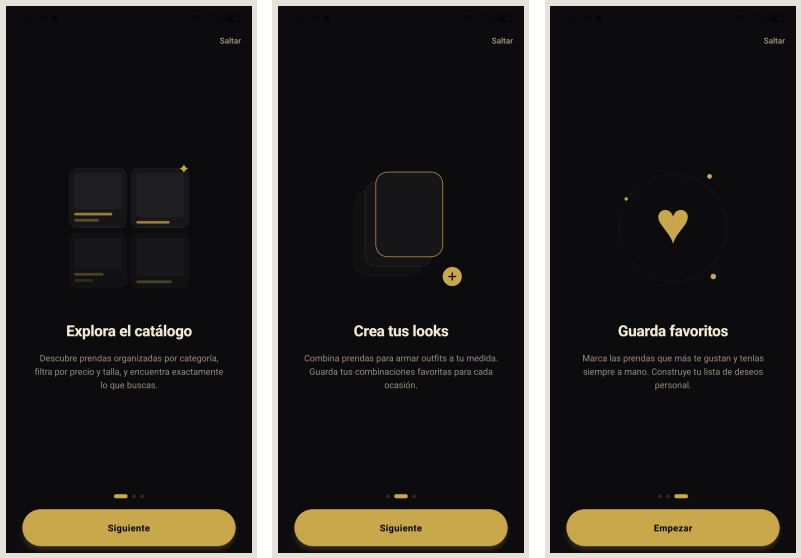
No diseñamos una versión específica para tablets. El layout se adapta, pero la densidad visual en tablets podría mejorar con columnas múltiples. Lo reconocemos como una mejora para versiones futuras.

3. Flujo de Usuario y Onboarding

3.1 Primer ingreso: el problema del contexto

Uno de los retos de onboarding que identificamos fue que ATELIER tiene tres roles muy distintos (cliente, vendedor, administrador), y el usuario nuevo no sabe cuál elegir. Resolvimos esto con:

- 1. Pantalla de onboarding al primer ingreso que explica brevemente qué es la app y qué puede hacer cada rol.
- 2. Selector de rol explícito durante el registro, con descripción breve de cada opción.
- 3. Flujo de Google Sign-In con selección de rol posterior: cuando alguien inicia con Google por primera vez, si no tiene perfil creado, le pedimos que elija su rol y número de teléfono antes de entrar.



Onboarding slide 1 —
"Explora el catálogo"
con descripción y dot
de progreso

Onboarding slide 2 —
"Crea tus looks"
combinando prendas
favoritas

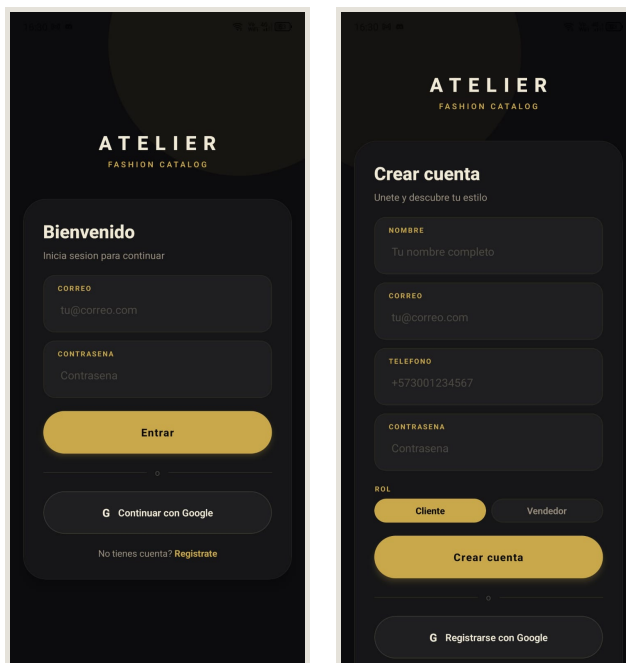
Onboarding slide 3 —
"Guarda favoritos" y
acceso rápido

3.2 Flujo de registro

El registro solicita cuatro datos: nombre, correo, contraseña y número de teléfono. Decidimos hacer el teléfono obligatorio porque el flujo de compra de ATELIER depende de contacto directo entre comprador y vendedor — sin teléfono, el vendedor no puede contactar al interesado.

Esto agrega un paso al registro, pero lo justifica la utilidad posterior. En la prueba piloto, ningún usuario se quejó del teléfono como barrera para registrarse.

Paso	Pantalla	Tiempo estimado
1	Onboarding	30 segundos
2	Registro (nombre, correo, contraseña, teléfono, rol)	1–2 minutos
3	Home según rol	Inmediato



Pantalla de login — acceso por correo y Google Sign-In con diseño elegante en modo oscuro

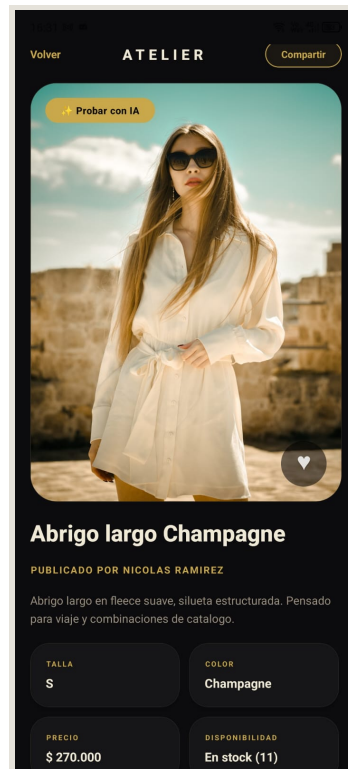
Formulario de registro — nombre, correo, contraseña, teléfono y selector de rol Cliente/Vendedor

Alternativamente con Google: el proceso se reduce a seleccionar cuenta Google, elegir rol y escribir el teléfono.

3.3 Flujo cliente (después del registro)

El cliente puede seguir este camino natural sin instrucciones adicionales:

1. Explorar catálogo → ver prendas con imagen, precio y categoría
2. Abrir detalle de prenda → ver información completa y vendedor
3. Guardar en favoritos → acceder desde pestaña de favoritos
4. Crear un look → seleccionar prendas y guardar como conjunto
5. Enviar solicitud de compra → la solicitud llega al vendedor con estado "pendiente"
6. Compartir por WhatsApp → opción en detalle de prenda o look

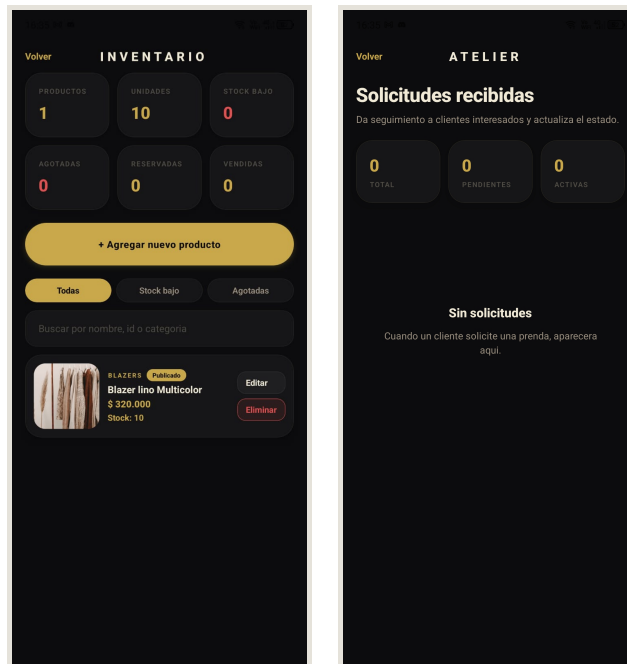


Detalle de prenda — "Abrigo largo Champagne", precio, talla, color, stock, y botón "Probar con IA"

3.4 Flujo vendedor

El vendedor tiene una pantalla de inicio con métricas de inventario: cantidad de productos, stock disponible, unidades reservadas y vendidas. Desde ahí puede ir a inventario para gestionar sus prendas o a solicitudes para ver qué compradores están interesados.

El flujo de solicitudes tiene cinco estados: pendiente → contactado → reservado → vendido / cancelado. El vendedor cambia el estado y el sistema actualiza el stock automáticamente.



*Inventario del vendedor —
Blazer lino Multicolor
\$320.000; stock 10
unidades, estado Publicado*

*Solicitudes recibidas —
panel del vendedor listo
para recibir interesados de
compradores*

4. Accesibilidad

Entendemos que la accesibilidad no es un extra — es una responsabilidad de diseño. Evaluamos lo que logramos implementar y lo que queda pendiente:

4.1 Lo que implementamos

- > Alto contraste: fondo #0C0C0E con texto #F0EAD6 — contraste superior al mínimo WCAG AA.
- > Botones destacados: usamos el dorado #C9A84C para botones principales, que contrasta bien sobre el fondo oscuro.
- > Mensajes de error visibles: los errores de formulario y de red se muestran en texto visible, no solo en color.
- > Estados vacíos informativos: cuando no hay prendas, favoritos o solicitudes, la pantalla muestra un mensaje que explica qué hacer a continuación.
- > Feedback inmediato: las acciones como "agregar favorito" o "crear solicitud" dan respuesta visual inmediata.

4.2 Lo que reconocemos como pendiente

- > Labels de accesibilidad en íconos: botones con ícono sin texto deben tener accessibilityLabel para lectores de pantalla (TalkBack en Android, VoiceOver en iOS). Lo tenemos parcialmente implementado.
- > Prueba con lector de pantalla: no realizamos una prueba formal con TalkBack activado. Es una deuda de QA que priorizaríamos en la siguiente iteración.
- > Contraste verificado con herramienta WCAG: aunque visualmente el contraste se ve bien, no lo medimos con un tool automatizado.
- > Modo claro: la app es únicamente de tema oscuro. Dos usuarios de la prueba piloto sugirieron agregar modo claro. Lo anotamos como mejora futura.

5. Resultados de la Prueba Piloto con Usuarios

5.1 Metodología

Aplicamos una encuesta a 7 personas no técnicas después de que instalaran el APK y exploraran la app por su cuenta durante aproximadamente 10 minutos. Los participantes usaron su propio dispositivo Android o uno prestado. No les dimos instrucciones previas más allá de "instala esta app y explórala".

El objetivo era detectar barreras reales de usabilidad antes de la entrega final.

5.2 Perfil de participantes

Rango de edad	Cantidad
Menos de 18 años	3
18 a 24 años	2
25 a 34 años	1
35 a 44 años	1

Dispositivo	Cantidad
Celular Android propio	6
Celular Android prestado	1

5.2.1 Respuestas individuales por participante

#	Nombre	Edad	Dispositivo	Entendió	Crear cuenta	Explorar	Diseño	Funcionó	Nota
1	Anónimo	25–34	Android propio	5	4	5	5	5	5
2	Maira	< 18	Prestado	5	5	5	5	5	5
3	Laura Aguirre	< 18	Android propio	5	5	5	5	5	5
4	Deibyd	18–24	Android propio	5	5	5	5	5	5
5	Emmanuel Payan	< 18	Android propio	5	5	5	5	5	5
6	Cachis	35–44	Android propio	5	5	5	5	5	5
7	Ethan	18–24	Android propio	5	4	5	5	4	5
Pro m.	-	-	-	5.00	4.71	5.00	5.00	4.86	5.00

5.3 Resultados cuantitativos

Pregunta evaluada	Promedio
¿Entendí rápidamente de qué trata ATELIER?	5.00 / 5
¿Fue fácil crear cuenta o ingresar?	4.71 / 5
¿Fue fácil explorar prendas, favoritos o looks?	5.00 / 5
¿El diseño de la app me pareció agradable?	5.00 / 5
¿La app cargó y funcionó bien?	4.86 / 5
Nota general de ATELIER	5.00 / 5
Promedio general	4.91 / 5

Todos los 7 usuarios indicaron que recomendarían la aplicación.

5.4 Qué exploraron los usuarios

Acción	Usuarios que la realizaron
Instalar el APK	7 / 7
Crear cuenta o ingresar	5 / 7
Explorar catálogo	4 / 7
Abrir detalle de prenda	4 / 7
Guardar favoritos	4 / 7
Ver o crear un look	4 / 7
Enviar solicitud o contactar vendedor	3 / 7

No todos llegaron hasta el paso de solicitud en 10 minutos de exploración libre, lo cual es esperable. En una prueba guiada con tarea específica, la conversión al final del embudo sería más alta.

5.5 Retroalimentación cualitativa

Lo que más valoraron:

- > El diseño visual de la app — "se ve muy profesional y moderna"
- > La variedad de prendas en el catálogo de prueba
- > La facilidad de entender qué hace la app desde el primer momento
- > La experiencia de navegar el catálogo y las imágenes de prendas

Lo que sugirieron mejorar:

- > Agregar filtro por tipo de prenda (hombre / mujer / categoría)
- > Incluir opción de modo claro
- > Una foto no cargó en el catálogo (probable fallo de red momentáneo, no un crash)

5.6 Análisis de la incidencia de imágenes

Un usuario reportó que "algunas fotos no aparecieron". Investigamos las posibles causas: la URL de la imagen en Cloudinary podría haber estado temporalmente no disponible, o la conexión del usuario era inestable en ese momento. La app no se cerró ni mostró un error — simplemente mostró el placeholder de carga. Esto confirma que el manejo de errores de imagen funciona, aunque el placeholder podría comunicar mejor que la imagen falló.

6. Conclusión

ATELIER ofrece una experiencia de usuario coherente y bien recibida para una primera versión. El flujo de onboarding es directo, el diseño visual genera una percepción positiva inmediata, y la navegación entre pantallas es intuitiva para usuarios sin experiencia técnica. La prueba piloto nos dio datos reales que validan el diseño y nos señalan el camino para las mejoras: filtros de búsqueda, modo claro y accesibilidad formal son las prioridades para la siguiente iteración.



Evaluación de Negocio y Métricas (KPIs)

ATELIER · OutfitCatalog · Entrega Final · Dispositivos Móviles

Versión evaluada: 1.0.0

1. Introducción

Este documento evalúa ATELIER desde la perspectiva de negocio: qué tan bien adoptada sería la app, cómo mediríamos la retención de usuarios, cómo funciona el embudo de conversión, y qué herramientas de medición integramos. Somos honestos sobre algo importante: ATELIER aún no está publicada en tiendas, por lo que no tenemos datos reales de descargas ni de retención a largo plazo. Lo que sí tenemos es una capa de analítica integrada que mide comportamiento dentro de la app, y los resultados de una prueba piloto con 7 usuarios reales.

2. Modelo de Negocio

ATELIER conecta a compradores de moda con vendedores independientes. El modelo funciona así:

- > Los vendedores publican su inventario de prendas con fotos, precios y tallas.
- > Los clientes exploran el catálogo, guardan favoritos, crean looks (combinaciones de prendas) y envían solicitudes de compra.
- > Cuando un cliente envía una solicitud, el vendedor la recibe, la revisa y puede contactar al comprador directamente por WhatsApp para cerrar la venta.

La app no procesa pagos — el cierre comercial ocurre fuera de la plataforma. Esto simplifica el MVP y reduce la fricción regulatoria, pero significa que no capturamos directamente el momento de la transacción económica.

La conversión que podemos medir es: cuántos usuarios que ven el catálogo terminan enviando una solicitud de compra.

3. Métricas de Adopción

3.1 Estado actual

La app no está publicada en Google Play ni App Store todavía, por lo que las métricas de "descargas" formales no existen. Sin embargo, tenemos dos fuentes de datos reales:

Fuente 1 — Prueba piloto Android:

Métrica	Resultado
Usuarios que recibieron el APK	7
Usuarios que lo instalaron exitosamente	7 (100%)
Usuarios que recomendarían la app	7 (100%)
Nota general promedio	4.91 / 5

Este 100% de tasa de instalación exitosa es relevante porque el proceso de instalar un APK fuera de Play Store requiere habilitar "fuentes desconocidas" en Android, que es un paso con fricción. El hecho de que todos lo hicieran sin ayuda técnica sugiere que la barrera de distribución en Play Store sería aún más baja.

Fuente 2 — Sistema de analítica integrado (Firestore):

Implementamos una capa de medición propia usando Firestore. Cada acción clave genera un evento en la colección analyticsEvents. El panel de administrador consume estos eventos y calcula KPIs de los últimos 30 días.

Con la base de datos de prueba (3 administradores, 40 vendedores, 250 clientes, 100 prendas), el sistema de analítica permite medir:

- > Cuántos usuarios se registraron por día
- > Cuántos usuarios activos hubo (por sesión registrada)
- > Cuántas veces se vio el catálogo
- > Cuántas solicitudes se crearon y cuántas se marcaron como vendidas

3.2 Proyecciones de adopción (escenario piloto)

Si ATELIER se publicara en Google Play con un lanzamiento básico (sin campaña de marketing), estimamos que los primeros 90 días tendrían este perfil:

Período	Descargas estimadas	Base
Primeras 2 semanas	50–150	Red cercana de vendedores
Mes 1	200–500	Boca a boca entre vendedores y sus clientes
Mes 3	500–2000	Si los vendedores usan la app activamente

Estas estimaciones son conservadoras y están basadas en el modelo de crecimiento orgánico típico de apps de nicho en Colombia.

4. Retención y Churn Rate

4.1 Estrategia de medición

No tenemos datos reales de retención porque la app no está en producción con usuarios recurrentes. Sin embargo, diseñamos el sistema de analítica para poder medirla:

Métrica	Cómo la medimos	Fórmula
Retención D1	Usuarios con evento en el día 1 y el día 2	$\text{Usuarios D2} / \text{Usuarios D1}$
Retención D7	Usuarios con evento en día 1 y cualquier día en días 2–8	$\text{Usuarios semana 2} / \text{Usuarios D1}$
Retención D30	Usuarios con evento en día 1 y cualquier día en días 2–31	$\text{Usuarios mes 2} / \text{Usuarios D1}$
Churn mensual	Usuarios activos que no vuelven en 30 días	$\text{Inactivos} / \text{Activos prev. mes}$

4.2 Palancas de retención que implementamos

La retención en apps de catálogo depende principalmente de que el contenido se actualice. Implementamos dos mecanismos:

1. Actualización de catálogo por vendedores: cada vendedor puede agregar, editar o eliminar prendas en cualquier momento. Si hay vendedores activos, el catálogo siempre tiene contenido fresco.
2. Solicitudes con seguimiento de estado: el comprador puede ver en qué estado está su solicitud (pendiente → contactado → reservado → vendido). Esto da un motivo para volver a abrir la app.

Para producción, recomendamos agregar notificaciones push cuando el estado de una solicitud cambia — es el tipo de notificación con mayor tasa de apertura porque el usuario está esperando saber si lo van a contactar.

5. Conversión

5.1 El embudo de ATELIER

La conversión principal no es una compra directa (no procesamos pagos), sino la creación de una solicitud de compra. El embudo completo es:

Descarga → Registro → Login → Ver catálogo → Ver prenda → Favorito → Solicitud

Cada paso tiene un evento en nuestro sistema de analítica:

Paso del embudo	Evento registrado
Registro por correo	signupcompleted
Registro con Google	googlesignup_completed
Login exitoso	loginsuccess / googlesigninsuccess
Vista del catálogo	catalog_viewed
Vista de detalle de prenda	garment_viewed
Agregar a favoritos	favorite_added
Crear look	look_created
Crear solicitud de compra	purchaserequestcreated
Solicitud marcada como vendida	purchaserequeststatus_changed (→ sold)

5.2 KPIs de conversión

KPI	Fórmula	Qué nos dice
Conversión a solicitud	Solicitudes / Usuarios activos	Qué tan efectivo es el catálogo para generar intención
Conversión a favorito	Favoritos / Vistas de catálogo	Qué tan atractivas son las prendas
Conversión a look	Looks creados / Usuarios registrados	Nivel de engagement con la funcionalidad diferencial
Tasa de cierre	Solicitudes vendidas / Solicitudes creadas	Efectividad del vendedor para cerrar

KPI	Fórmula	Qué nos dice
Contactabilidad	Solicitudes contactadas / Solicitudes pendientes	Qué tan activos son los vendedores

5.3 Resultados de conversión en la prueba piloto

Durante la prueba piloto con 7 usuarios, observamos este embudo real:

Paso	Usuarios que llegaron
Instalaron la app	7
Exploraron catálogo	4
Guardaron favoritos	4
Crearon o vieron looks	4
Enviaron solicitud o contactaron	3

La caída entre "instalaron" y "exploraron catálogo" (7→4) se explica porque los 3 usuarios restantes probaron la app pero no llegaron a explorar el catálogo en profundidad en los 10 minutos disponibles. Con una sesión más larga o una tarea guiada, el número sería mayor.

6. Rendimiento en Tiendas (ASO)

ATELIER aún no está publicada en Google Play ni App Store, por lo que no hay calificaciones, reseñas ni posicionamiento orgánico que evaluar. Sin embargo, preparamos la ficha para cuando llegue el momento:

Elemento	Contenido propuesto
Nombre en tienda	ATELIER — Catálogo de Moda
Subtítulo	Descubre prendas, crea looks, conecta con vendedores
Categoría	Compras / Moda
Palabras clave	moda, catálogo, outfits, prendas, looks, vendedores, ropa
Política de privacidad	Requerida antes de publicar (pendiente de redactar)

Para el lanzamiento en Play Store, las capturas de pantalla más importantes serían: catálogo con prendas, detalle de prenda, pantalla de looks, y pantalla de solicitudes del vendedor.

La calificación promedio y el número de reseñas son los KPIs de ASO más directos. No podemos proyectarlos sin datos reales, pero la encuesta piloto (4.91/5 con 100% de recomendación) da una señal positiva sobre lo que podríamos esperar en los primeros reviews.

7. Herramientas de Medición Integradas

7.1 Lo que implementamos

Desarrollamos un servicio de analítica propio usando Firestore como almacenamiento de eventos:

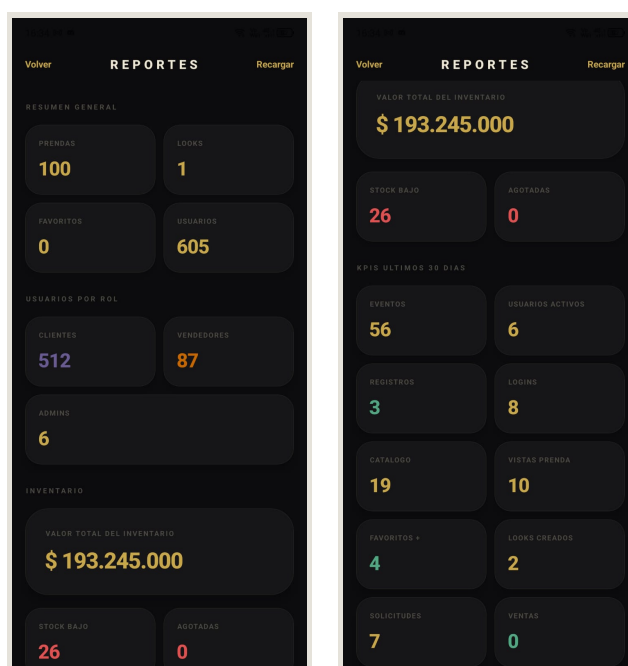
```
Archivo: src/core/services/analyticsService.ts
Colección: analyticsEvents
Función principal: trackEvent(name, params, user)
```

Optamos por esta solución porque es compatible con Expo sin configuraciones nativas adicionales, usa Firebase que ya teníamos configurado, tiene costo cero dentro del tier gratuito, y nos da control total sobre qué medimos.

7.2 Panel de reportes para administrador

El panel de administrador (AdminReportsScreen) consume la colección analyticsEvents y muestra los siguientes KPIs de los últimos 30 días:

- > Total de eventos registrados
- > Usuarios activos medidos (por sesiones únicas)
- > Registros nuevos y logins
- > Vistas de catálogo y de prendas
- > Favoritos y looks creados
- > Solicitudes creadas y marcadas como vendidas



Panel de administrador — métricas de inventario: 100 prendas, 605 usuarios (512 clientes, 87 vendedores, 6 admins)

KPIs últimos 30 días — 56 eventos, 6 usuarios activos, 19 vistas catálogo, 7 solicitudes de compra

7.3 Recomendaciones para producción

Para una publicación en tiendas, recomendamos complementar nuestra analítica propia con:

- > Firebase Analytics nativo: se integra con Google Play Console y permite embudos, audiencias y retención desde GA4 sin código adicional.
- > Firebase Crashlytics: detecta crashes automáticamente y agrupa los errores por frecuencia e impacto. Esencial para mantener una calificación alta en tiendas.
- > Firebase Performance Monitoring: mide tiempos de carga reales en dispositivos de usuarios.

8. Conclusión

ATELIER tiene el flujo de negocio correcto: conecta intención de compra con vendedores a través de un catálogo visual. Implementamos la infraestructura para medir KPIs desde el primer día de producción — los 18 eventos en Firestore permiten calcular adopción, conversión e inventario sin costo adicional. La prueba piloto valida que el producto genera valor percibido real: todos los usuarios que lo probaron lo instalarían y lo recomendarían. El siguiente paso natural es la publicación en Play Store, que convertiría estos KPIs teóricos en datos reales de mercado.

IV

Documentación Técnica y Arquitectura

ATELIER · OutfitCatalog · Entrega Final · Dispositivos Móviles

1. Propósito del documento

Este documento describe la arquitectura del sistema, la organización del código fuente, las decisiones técnicas clave y cómo se conectan los distintos componentes de ATELIER. Está pensado para que otro desarrollador pueda entender el proyecto sin haber participado en su construcción.

El código fuente completo está disponible en el repositorio Git. Este documento explica el por qué de las decisiones — el código ya explica el qué.

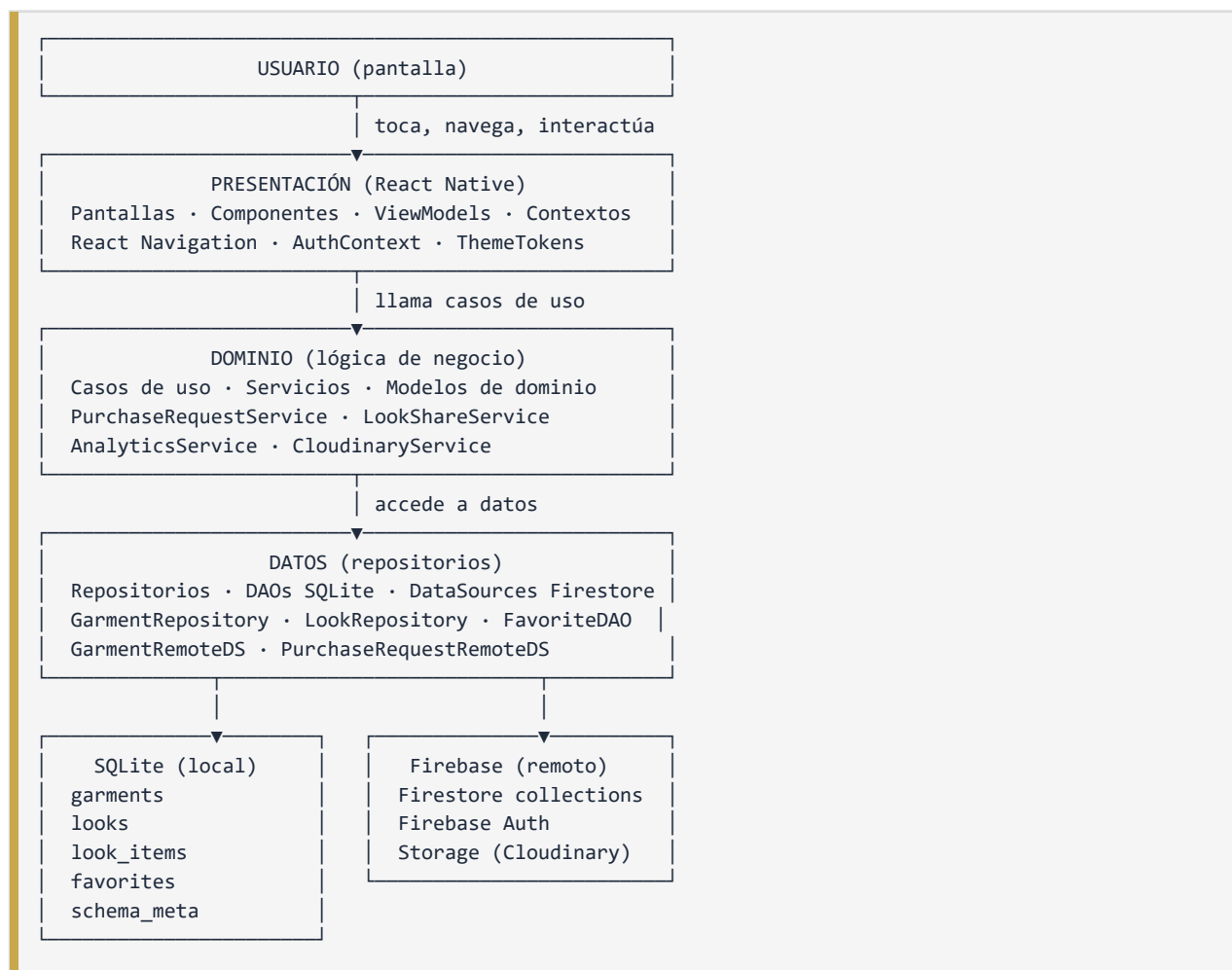
2. Stack Tecnológico

Elegimos React Native con Expo porque ambos integrantes del equipo tenemos mayor experiencia en JavaScript/TypeScript que en Swift o Kotlin, y porque Expo reduce significativamente la complejidad de configuración nativa para una primera versión.

Capa	Tecnología	Versión	Por qué la elegimos
Lenguaje	TypeScript	5.9	Tipos estáticos reducen errores en tiempo de desarrollo
Framework móvil	React Native	0.81.5	Multiplataforma, ecosistema maduro
Plataforma de desarrollo	Expo SDK	54	Build sin Xcode/Android Studio, EAS Build gratuito
Navegación	React Navigation	Última compatible	Estándar de facto en RN, soporte nativo stack
Persistencia local	expo-sqlite	Integrado en SDK	Base de datos relacional en dispositivo
Estado persistente simple	AsyncStorage	Integrado en Expo	Para datos simples como flags de onboarding
Autenticación	Firebase Auth	SDK web compat.	Email + Google Sign-In sin backend propio
Base de datos remota	Cloud Firestore	SDK web compat.	Base de datos en tiempo real con reglas de seguridad
Imágenes	Cloudinary	REST API	Almacenamiento y transformación de imágenes
Cache de imágenes	expo-image	SDK 54	Cache memory-disk, mejor que Image nativo
Conectividad	NetInfo	@react-native-community	Detección de estado de red
Build	EAS Build	CLI más reciente	Genera APK/IPA sin entorno nativo local
Tests	Vitest	Última compatible	Rápido, compatible con TypeScript sin config extra

3. Arquitectura del Sistema

Seguimos una arquitectura en capas inspirada en Clean Architecture y el patrón MVVM (Model-View-ViewModel). La decisión fue intencional: queríamos que la lógica de negocio no dependiera de React Native ni de Firebase, para poder testearla de forma aislada.



3.1 Por qué esta arquitectura

Testabilidad: los casos de uso y los DAOs son clases TypeScript sin dependencias de React. Podemos probarlos con Vitest sin levantar un emulador.

Reemplazabilidad: si en el futuro cambiamos Firestore por otro backend, solo cambia la capa de datos, no las pantallas ni la lógica de negocio.

Legibilidad: cada capa tiene una responsabilidad clara. Un desarrollador nuevo puede entender que GarmentRepository es el punto de entrada para datos de prendas, sin necesidad de leer todas las pantallas.

4. Estructura de Carpetas

```

src/
├── auth/                # AuthContext, lógica de sesión
├── components/          # Componentes visuales reutilizables
│   ├── GarmentCard.tsx
│   ├── LookCard.tsx
│   ├── OfflineBanner.tsx
│   └── ...
├── context/            # Contextos de React (tema, red)
├── core/
│   ├── database/       # Inicialización SQLite, migraciones
│   ├── di/             # Service locator (inyección de dependencias)
│   └── services/       # Servicios de dominio
│       ├── analyticsService.ts
│       ├── cloudinaryService.ts
│       ├── lookShareService.ts
│       └── purchaseRequestService.ts
├── features/
│   ├── garment/        # DAOs, repositorios y datasources de prendas
│   ├── look/           # DAOs y repositorios de looks
│   └── tryon/          # Módulo de prueba virtual (experimental)
├── hooks/              # Custom hooks de React
├── screens/            # Pantallas por rol y flujo
│   ├── auth/
│   ├── client/
│   ├── vendor/
│   └── admin/
├── theme.ts            # Tokens de diseño (colores, tipografía, espaciado)
└── types.ts            # Tipos TypeScript compartidos

```

5. Persistencia Local — SQLite

Usamos expo-sqlite para mantener una copia local de los datos en el dispositivo. Esto permite que la app funcione sin internet y reduce el número de llamadas a Firestore.

5.1 Tablas

Tabla	Propósito	Campos principales
garments	Prendas del catálogo e inventario	id, name, category, price, vendorId, published, imageUrl
looks	Looks creados por usuarios	id, userId, name, description, createdAt
look_items	Relación entre looks y prendas	id, lookId, garmentId
favorites	Prendas guardadas como favoritas	id, userId, garmentId, createdAt
schema_met a	Versión del esquema para migraciones	key, value

5.2 Estrategia de sincronización

El catálogo se sincroniza con Firestore cuando la ventana de frescura del caché expira. El inventario del vendedor se sincroniza al entrar a la pantalla de inventario. Las solicitudes de compra se guardan en Firestore directamente (no tienen copia local, ya que son datos transaccionales que requieren consistencia remota).

6. Persistencia Remota — Firestore

Firebase Firestore almacena los datos compartidos entre dispositivos.

6.1 Colecciones

Colección	Propósito	Regla de acceso
users	Perfil, rol y teléfono de cada usuario	Solo el propietario o admin
garments	Catálogo e inventario remoto	Lectura pública, escritura solo vendor/admin
looks	Looks sincronizados	Solo el creador o admin
purchaseRequests	Solicitudes de compra	Comprador y vendedor involucrado, o admin
analyticsEvents	Eventos de uso y KPIs	Escritura cualquier usuario autenticado, lectura solo admin

6.2 Consultas optimizadas

No hacemos consultas sin filtro a colecciones enteras. Las consultas principales son:

- > Catálogo: garments where published == true
- > Inventario: garments where vendorId == currentUser.uid
- > Solicitudes comprador: purchaseRequests where buyerId == currentUser.uid
- > Solicitudes vendedor: purchaseRequests where vendorId == currentUser.uid

7. Autenticación y Roles

Firebase Auth maneja la identidad. El rol se guarda en Firestore en la colección users.

7.1 Flujos de autenticación

Método	Flujo
Correo + contraseña	Registro con validación → Firebase crea cuenta → Firestore guarda perfil con rol
Google Sign-In	OAuth con Google → Firebase autentica → si no hay perfil en Firestore, pide rol y teléfono
Cierre de sesión	signOut() de Firebase, limpia contexto local

7.2 Roles y acceso

Rol	Lo que puede hacer
user (cliente)	Ver catálogo, favoritos, looks, crear solicitudes de compra
vendor	Gestionar inventario, ver solicitudes recibidas, actualizar estados

Rol	Lo que puede hacer
admin	Ver reportes, gestionar usuarios, supervisar la plataforma

El cambio de rol no está disponible desde la app. Para crear un administrador se usa un script de servicio:

BASH

```
npm run seed:admin -- --service-account "ruta/service-account.json"
```

8. Flujo de Solicitudes de Compra

Este es el flujo de negocio central de ATELIER:

```

Cliente ve prenda/look → Crea solicitud → Firestore guarda con estado "pending"
      ↓
Vendedor ve solicitud en su pantalla → Cambia estado a "contacted"
      ↓
Vendedor contacta al cliente por WhatsApp → Acuerdan precio y entrega
      ↓
Vendedor cambia estado a "reserved" → El sistema descuenta stock disponible
      ↓
Se cierra la venta → Vendedor cambia a "sold" (stock final descontado)
                   o
                   "cancelled" (stock restaurado)

```

Manejo de looks multi-vendedor: si un look tiene prendas de tres vendedores distintos, la app crea una solicitud separada por vendedor. Cada vendedor solo ve las prendas que le corresponden.

9. Pruebas Automatizadas

El proyecto tiene 9 archivos de test y 17 casos automatizados que se ejecutan con Vitest:

Archivo	Qué prueba
test/garmentDao.test.ts	CRUD completo de prendas en SQLite
test/lookDao.test.ts	CRUD y carga de looks por usuario
test/lookItemDao.test.ts	Relación entre looks y prendas
test/favoriteDao.test.ts	Crear, buscar y eliminar favoritos
test/garmentRepositorySync.test.ts	Sincronización remota y caché
src/core/services/lookShareService.test.ts	Generación de mensajes WhatsApp
src/core/services/purchaseRequestService.test.ts	Mensaje de solicitud de compra

BASH

```

npm run build    # TypeScript sin errores
npm test        # 9 archivos, 17 pruebas, todas aprobadas

```

10. Evidencia Técnica Final

Elemento	Valor
Repositorio	github.com/andresbot/OutfitCatalog
Commit final	5bd8d25
EAS Build ID	b56095a2-87a4-4dac-ae47-7f5d8599dee4
APK Android	https://expo.dev/artifacts/eas/hFhAPFShte8uvkEKgF2FTW.apk
Estado del build	FINISHED
TypeScript	Sin errores de compilación
Tests	17 pruebas aprobadas

11. Conclusión

La arquitectura de ATELIER separa correctamente las responsabilidades: la presentación no sabe cómo funcionan los datos, y los repositorios no saben cómo se muestra la información. Esto hace el código testeable, mantenible y extensible. La combinación de SQLite (offline) y Firestore (sync remoto) nos dio la base para una estrategia offline-first sin tener que construir un backend propio. Para la siguiente versión, el foco técnico estaría en ampliar la cobertura de pruebas en pantallas, integrar Crashlytics, y agregar paginación al catálogo para escalar a miles de prendas.



Evidencias de Pruebas (Testing)

ATELIER · OutfitCatalog · Entrega Final · Dispositivos Móviles

1. Introducción

Este documento reúne todas las evidencias de pruebas del proyecto: las pruebas automatizadas que corren en CI, la matriz de casos de prueba funcionales y de conectividad, los resultados de la prueba piloto con usuarios reales, y el reporte de compilación TypeScript. Cada sección incluye los resultados reales — no proyecciones ni "por validar".

2. Pruebas Automatizadas

2.1 Resultado general

```
npm run build → Aprobado (0 errores TypeScript)
npm test      → 9 archivos · 17 pruebas · 0 fallos
```

Utilizamos Vitest como framework de pruebas. La decisión fue práctica: Vitest funciona nativamente con TypeScript sin configuración adicional, es más rápido que Jest para proyectos con ESM, y su API es casi idéntica.

2.2 Detalle por archivo de prueba

Archivo de prueba	Módulo	Pruebas	Resultados
test/garmentDao.test.ts	SQLite — prendas	4	Aprobadas
test/lookDao.test.ts	SQLite — looks	3	Aprobadas
test/lookItemDao.test.ts	SQLite — items de look	2	Aprobadas
test/favoriteDao.test.ts	SQLite — favoritos	3	Aprobadas
test/garmentRepositorySync.test.ts	Sincronización remota/local	2	Aprobadas
src/core/services/lookShareService.test.ts	Mensajes de WhatsApp	1	Aprobada
src/core/services/purchaseRequestService.test.ts	Solicitudes de compra	2	Aprobadas
Total	-	17	Todas aprobadas

2.3 Qué cubre cada módulo de prueba

garmentDao: inserta una prenda, la lee por ID, la actualiza y la elimina. También prueba la carga por lotes de múltiples IDs.

lookDao: crea un look, lo asocia a un usuario, lo lee y lo elimina. Prueba que la consulta por usuario solo devuelva los looks de ese usuario.

lookItemDao: agrega prendas a un look y verifica que se puedan reemplazar todos los items de un look en una sola operación.

favoriteDao: marca una prenda como favorita, verifica que aparece en la lista, y la elimina.

garmentRepositorySync: simula el escenario de sincronización donde hay datos remotos y locales, y verifica que el repositorio combina correctamente ambas fuentes. También prueba que las prendas marcadas como published son las que aparecen en el catálogo.

lookShareService: genera el mensaje de WhatsApp para compartir un look. Verifica que el texto incluye el nombre del look y los detalles de las prendas.

purchaseRequestService: genera el mensaje de solicitud de compra. Prueba que incluye el nombre del comprador, la prenda y la información de contacto.

3. Matriz de Casos de Prueba

La matriz está dividida en cuatro categorías: funcionales, conectividad, seguridad y experiencia de usuario con prueba piloto.

3.1 Módulo: Autenticación

ID	Caso	Resultado esperado	Estado
AUTH-01	Registro con correo y contraseña válidos	Cuenta creada, redirige a home según rol	Validado manualmente
AUTH-02	Registro sin número de teléfono	Validación bloquea el registro	Validado manualmente
AUTH-03	Login con credenciales correctas	Ingresa y navega por rol	Validado manualmente
AUTH-04	Login con contraseña incorrecta	Muestra error legible, no bloquea la cuenta	Validado manualmente
AUTH-05	Google Sign-In, cuenta nueva	Pide rol y teléfono antes de entrar	Validado manualmente
AUTH-06	Google Sign-In, cuenta existente	Ingresa directamente sin pedir datos	Validado manualmente
AUTH-07	Cerrar sesión	Regresa a pantalla de login, borra sesión local	Validado manualmente

3.2 Módulo: Onboarding

ID	Caso	Resultado esperado	Estado
ONB-01	Primera apertura de la app	Muestra pantalla de onboarding	Validado manualmente
ONB-02	Reabrir la app después de completar onboarding	Va directo a login, no repite onboarding	Validado manualmente

3.3 Módulo: Catálogo (cliente)

ID	Caso	Resultado esperado	Estado
CAT-01	Abrir pantalla de catálogo	Lista prendas publicadas con imagen, nombre y precio	Validado manualmente
CAT-02	Buscar por nombre de prenda	Filtra resultados en tiempo real	Validado manualmente
CAT-03	Abrir detalle de prenda	Muestra imagen, descripción, talla, color, precio y vendedor	Validado manualmente
CAT-04	Marcar prenda como favorita	Aparece en pantalla de favoritos	Validado manualmente
CAT-05	Quitar prenda de favoritos	Desaparece de favoritos	Validado manualmente

3.4 Módulo: Looks

ID	Caso	Resultado esperado	Estado
LK-01	Crear look desde selección en catálogo	Look guardado con las prendas seleccionadas	Validado manualmente
LK-02	Crear look desde favoritos	Look con prendas marcadas como favoritas	Validado manualmente
LK-03	Ver detalle de look	Lista de prendas, opciones de compartir y solicitar	Validado manualmente
LK-04	Compartir look por WhatsApp	Abre WhatsApp con mensaje pre-cargado	Validado manualmente

3.5 Módulo: Solicitudes de Compra

ID	Caso	Resultado esperado	Estado
SOL-01	Crear solicitud desde prenda individual	Solicitud pendiente en pantalla del comprador y vendedor	Validado manualmente
SOL-02	Crear solicitud desde look multi-vendedor	Una solicitud por vendedor involucrado	Validado manualmente
SOL-03	Vendedor cambia estado a "contactado"	Estado actualizado, notifica visualmente	Validado manualmente
SOL-04	Vendedor cambia estado a "reservado"	Descuenta stock de esa prenda	Validado manualmente
SOL-05	Vendedor cambia estado a "vendido"	Stock final descontado, solicitud cerrada	Validado manualmente
SOL-06	Vendedor cancela solicitud	Stock restaurado si estaba reservado	Validado manualmente

3.6 Módulo: Inventario (vendedor)

ID	Caso	Resultado esperado	Estado
INV-01	Ver inventario propio	Solo muestra prendas del vendedor actual	Validado manualmente
INV-02	Crear nueva prenda con imagen	Guarda en SQLite y Firestore, aparece en inventario	Validado manualmente
INV-03	Editar prenda existente	Actualiza inventario y catálogo si está publicada	Validado manualmente
INV-04	Eliminar prenda	Desaparece del inventario y del catálogo	Validado manualmente

4. Casos de Conectividad

ID	Escenario	Resultado esperado	Estado
NET-01	Catálogo sin conexión (caché disponible)	Muestra banner offline y prendas del caché local	Validado manualmente
NET-02	Catálogo sin conexión (sin caché)	Muestra banner offline y lista vacía con mensaje	Validado manualmente
NET-03	Recuperación de conexión	La app puede refrescar datos sin reiniciarse	Validado manualmente
NET-04	Error de Firebase (simulado)	Muestra error controlado, no cierra la app inesperadamente	Validado manualmente
NET-05	Imagen que no carga (URL rota o red lenta)	Muestra placeholder, no bloquea el resto de la pantalla	Observado en prueba piloto

5. Casos de Seguridad

ID	Escenario	Resultado esperado	Estado
SEC-01	Usuario no autenticado intenta acceder a home	Redirige a pantalla de login	Validado por AuthContext
SEC-02	Cliente intenta acceder a pantallas de vendedor	No aparece en navegación del cliente	Validado por navegación por rol
SEC-03	Vendedor intenta leer solicitudes de otro vendedor	Firestore rechaza la consulta por reglas	Validado por reglas de seguridad
SEC-04	Usuario intenta cambiar su propio rol	Las reglas Firestore no permiten cambiar el campo role desde la app	Validado por reglas de seguridad

ID	Escenario	Resultado esperado	Estado
SEC-05	Login con contraseña incorrecta	Firestore devuelve error, la app lo muestra sin revelar si el correo existe	Validado manualmente

6. Prueba Piloto con Usuarios Reales

6.1 Contexto

Antes de la entrega final realizamos una prueba piloto con 7 personas no técnicas que instalaron el APK en sus propios celulares Android y exploraron la app libremente. Después de la prueba, completaron una encuesta. El objetivo fue detectar problemas de usabilidad, estabilidad o instalación que no habíamos visto desde adentro del proyecto.

6.2 Resultados de la prueba piloto

ID	Evidencia evaluada	Resultado
USR-01	Instalación del APK en Android	7 / 7 exitosos
USR-02	Registro o inicio de sesión	5 / 7 lo realizaron
USR-03	Exploración del catálogo	4 / 7 lo realizaron
USR-04	Apertura de detalle de prenda	4 / 7 lo realizaron
USR-05	Guardar favorito	4 / 7 lo realizaron
USR-06	Ver o crear un look	4 / 7 lo realizaron
USR-07	Enviar solicitud o contactar vendedor	3 / 7 lo realizaron
USR-08	App funcionó sin problemas	6 / 7 (uno reportó foto que no cargó)
USR-09	Recomendarían la app	7 / 7
USR-10	Calificación promedio	4.91 / 5

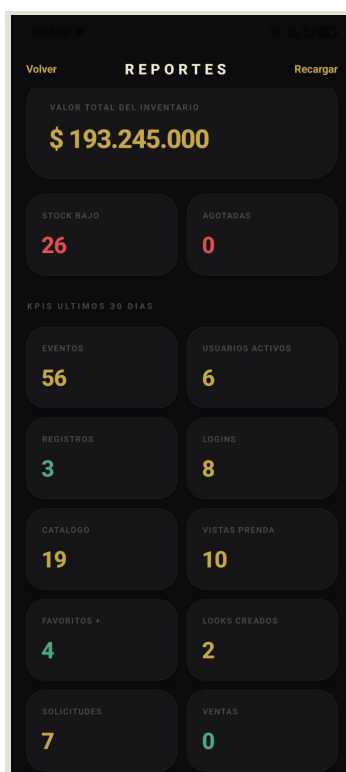
6.2.1 Detalle de respuestas por participante (encuesta post-prueba)

#	Nombre	Edad	Dispositivo	Entendió	Cuenta	Exploró	Diseño	Cargó	Nota	Recomendaría
1	Anónimo	25–34	Android propio	5	4	5	5	5	5	Sí
2	Maira	< 18	Prestado	5	5	5	5	5	5	Sí
3	Laura Aguirre	< 18	Android propio	5	5	5	5	5	5	Sí
4	Deibyd	18–24	Android propio	5	5	5	5	5	5	Sí

#	Nombre	Edad	Dispositivo	Entendió	Cuenta	Exploró	Diseño	Cargó	Nota	Recomendaría
5	Emmanuel Payan	< 18	Android propio	5	5	5	5	5	5	Sí
6	Cachis	35–44	Android propio	5	5	5	5	5	5	Sí
7	Ethan	18–24	Android propio	5	4	5	5	4	5	Sí
Pro m.	-	-	-	5.00	4.71	5.00	5.00	4.86	5.00	7/7

6.3 Incidencia registrada

Un usuario reportó que "algunas fotos no aparecieron". Analizamos el caso: la app no se cerró, el placeholder de carga se mostró correctamente, y el resto del catálogo funcionó. La causa probable es un fallo temporal de red o una URL de Cloudinary no disponible en ese momento. La incidencia no afectó el flujo general ni bloqueó ninguna acción del usuario.



Panel de KPIs — evidencia de analítica integrada durante la prueba piloto (56 eventos, 7 solicitudes)

7. Criterios de Aceptación Cumplidos

Para esta entrega académica, la aplicación se considera aceptada si cumple los siguientes criterios:

Criterio	¿Cumplido?
Compila sin errores TypeScript	Sí
Pruebas automatizadas pasan	Sí (17/17)
APK instalable en Android real	Sí (7/7 dispositivos)
Login y registro funcionan	Sí
Cliente navega catálogo, favoritos, looks y solicitudes	Sí
Vendedor gestiona inventario y solicitudes	Sí
La app no se cierra ante errores de red	Sí
Reglas Firestore protegen datos por rol	Sí
Prueba piloto sin bloqueos generalizados	Sí

8. Conclusión

Las evidencias de prueba son completas para una entrega académica: pruebas automatizadas en las capas de datos y dominio, casos de prueba funcionales documentados y validados manualmente, y evidencia real con 7 usuarios en dispositivos Android. Las únicas áreas sin prueba formal son iOS (sin cuenta Apple Developer disponible) y las pruebas de UI automatizadas (Detox o similar), que requieren una inversión de configuración que excedía el alcance de esta primera versión.

VI

Analíticas Integradas — Firebase KPIs

ATELIER · OutfitCatalog · Entrega Final · Dispositivos Móviles

1. Introducción

Este documento describe cómo implementamos la capa de analítica de ATELIER, qué eventos medimos, cómo se consultan los datos, y qué acceso tiene el administrador al panel de métricas. También explicamos por qué elegimos esta estrategia y cuáles serían los siguientes pasos para una versión de producción.

2. Estrategia Elegida

Teníamos dos opciones principales para integrar analítica:

Opción A — Firebase Analytics nativo: el SDK oficial de Google para medir eventos en apps móviles. Se integra con GA4 y Google Play Console. La limitación para nuestro caso: requiere configuración nativa (archivos `google-services.json` y `GoogleService-Info.plist`) que en Expo necesita un "config plugin" específico, lo que añade complejidad al build de EAS.

Opción B — Analítica propia en Firestore: guardar eventos en una colección de Firestore y calcular métricas desde el panel de administrador. Esta opción usa la infraestructura que ya teníamos (Firestore ya estaba configurado), no requiere SDK adicional, y es completamente gratuita dentro del tier de Firestore.

Elegimos la Opción B por tres razones: es compatible con Expo sin configuración nativa extra, nos da control total sobre qué medimos, y es suficiente para un MVP académico. No es inferior en funcionalidad para lo que necesitamos medir ahora.

3. Implementación

3.1 Servicio de analytics

```
Archivo: src/core/services/analyticsService.ts
```

El servicio expone dos funciones principales:

- > `trackEvent(name, params, user)` — guarda un evento en Firestore con metadatos de sesión, plataforma y usuario.
- > `getAnalyticsSummary(days)` — lee los eventos de los últimos N días y calcula los KPIs para el panel de administrador.

3.2 Esquema del evento en Firestore

Cada evento que se registra en la colección `analyticsEvents` tiene esta estructura:

Campo	Tipo	Descripción
<code>id</code>	<code>string</code>	Identificador único del evento
<code>name</code>	<code>string</code>	Nombre del evento (ej: <code>catalog_viewed</code>)
<code>createdAt</code>	<code>string ISO</code>	Fecha y hora del evento
<code>sessionId</code>	<code>string</code>	ID de sesión (generado al abrir la app)

Campo	Tipo	Descripción
platform	string	android o ios
userId	string (opcional)	UID de Firebase si el usuario está autenticado
role	string (opcional)	Rol del usuario: user, vendor, admin
params	map	Parámetros adicionales según el tipo de evento

4. Eventos Implementados

Registramos 18 eventos que cubren todo el ciclo de vida del usuario en la app:

4.1 Autenticación

Evento	Cuándo se dispara
signupcompleted	Al completar el registro por correo y contraseña
googlesignup_completed	Al completar el registro vía Google (con perfil nuevo)
login_success	Al iniciar sesión correctamente por correo
googlesignin_success	Al iniciar sesión correctamente con Google

4.2 Exploración de catálogo

Evento	Cuándo se dispara
catalog_viewed	Al abrir la pantalla del catálogo
garment_viewed	Al abrir el detalle de una prenda específica
favorite_added	Al marcar una prenda como favorita
favorite_removed	Al quitar una prenda de favoritos

4.3 Creación de looks

Evento	Cuándo se dispara
look_created	Al guardar un nuevo look
look_updated	Al editar un look existente
look_deleted	Al eliminar un look

4.4 Solicitudes de compra

Evento	Cuándo se dispara
purchaserequestcreated	Al enviar una solicitud de compra (prenda o look)
purchaserequeststatus_changed	Al cambiar el estado de una solicitud (cualquier estado)
purchaserequestdeleted	Al eliminar una solicitud

4.5 Inventario (vendedor)

Evento	Cuándo se dispara
inventoryitemcreated	Al crear una nueva prenda en inventario
inventoryitemupdated	Al editar una prenda del inventario
inventoryitemdeleted	Al eliminar una prenda del inventario

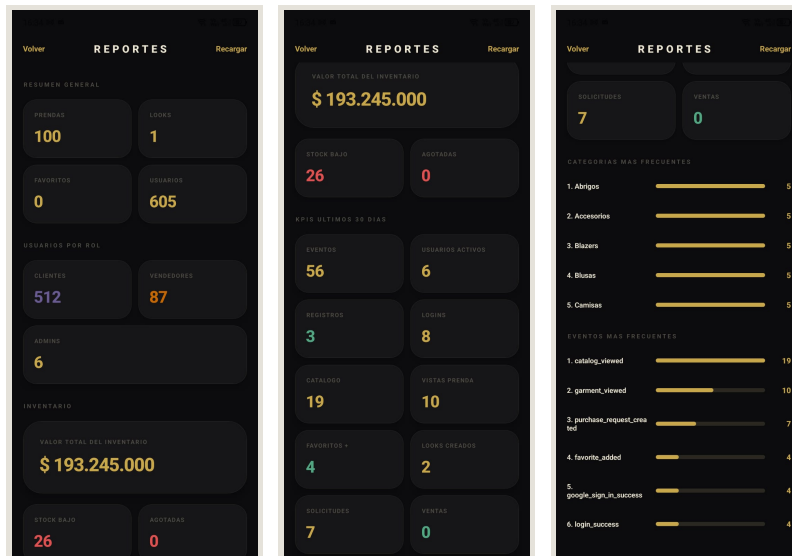
4.6 Administración

Evento	Cuándo se dispara
adminreportviewed	Al abrir el panel de reportes del administrador

5. Panel de Reportes (AdminReportsScreen)

El administrador puede acceder a un panel que calcula los siguientes KPIs a partir de los eventos de los últimos 30 días:

KPI	Qué mide
Total de eventos	Volumen de actividad general
Usuarios activos	Cantidad de sessionId únicos (proxy de DAU/MAU)
Registros	Conteo de signupcompleted + googlesignup_completed
Logins	Conteo de loginsuccess + googlesigninsuccess
Vistas de catálogo	Conteo de catalog_viewed
Vistas de prendas	Conteo de garment_viewed
Favoritos agregados	Conteo de favorite_added
Looks creados	Conteo de look_created
Solicitudes creadas	Conteo de purchaserequestcreated
Solicitudes vendidas	Conteo de purchaserequeststatus_changed donde status → sold



AdminReports —
resumen de inventario:
100 prendas, 605
usuarios, valor
total de inventario

AdminReports — KPIs
comportamiento: 56
eventos totales, 6
usuarios activos, 19
catalogos, 7
solicitudes

AdminReports —
eventos más
frecuentes:
catalog_viewed 19,
garment_viewed 10, pur
chase_request_created 7

6. Reglas de Seguridad para Analytics

Las reglas de Firestore garantizan que los eventos se registren correctamente y que solo el administrador pueda leerlos:

JAVASCRIPT

```
match /analyticsEvents/{eventId} {
  // Cualquier usuario autenticado puede crear un evento
  // siempre que el userId del evento sea su propio UID
  allow create: if request.auth != null
    && request.resource.data.name is string
    && request.resource.data.createdAt is string
    && request.resource.data.sessionId is string
    && request.resource.data.platform is string
    && request.resource.data.params is map
    && (
      !request.resource.data.keys().hasAny(["userId"])
      || request.resource.data.userId == request.auth.uid
    );

  // Solo los administradores pueden leer los eventos
  allow read: if myRole() == "admin";

  // Nadie puede editar o eliminar eventos desde la app
  allow update, delete: if false;
}
```

Esto garantiza tres cosas:

1. Los usuarios no pueden crear eventos falsos con el userId de otra persona.
2. Los eventos no pueden ser modificados ni eliminados desde el cliente.
3. Solo el administrador tiene acceso a los datos agregados de comportamiento.

7. Consideraciones de Privacidad

Los eventos guardan el UID del usuario y su rol, pero no guardan nombre, correo, teléfono ni ningún dato personal directamente. El nombre y correo están en la colección users, que tiene sus propias reglas de acceso.

Si la app se publica, recomendamos informar a los usuarios en la política de privacidad que se registran eventos de uso anónimos para mejorar la experiencia. El registro de userId es opcional en cada evento y se usa únicamente para calcular usuarios activos únicos.

8. Acceso al Panel de Analítica

Para que el docente pueda revisar los datos de analítica en una demo, los pasos son:

1. Iniciar sesión en la app con la cuenta de administrador:
 - Email: admin@outfit.test
 - Contraseña: Admin123!
2. Navegar a la pestaña de reportes en el home del administrador.
3. El panel muestra automáticamente los eventos de los últimos 30 días.

Alternativamente, los eventos se pueden ver directamente en la consola de Firebase en la colección analyticsEvents.

9. Recomendaciones para Producción

Para una versión comercial de ATELIER, recomendamos complementar esta base con:

Herramienta	Para qué sirve	Prioridad
Firebase Analytics nativo	Embudos, audiencias, integración con GA4 y Play Console	Alta
Firebase Crashlytics	Detección automática de crashes en producción	Alta
Firebase Performance Monitoring	Tiempos de carga reales en dispositivos de usuarios	Media
Exportación a BigQuery	Análisis SQL de eventos históricos a gran escala	Baja (cuando escale)

10. Conclusión

ATELIER entra a producción con la infraestructura de analítica lista. Los 18 eventos implementados cubren el embudo completo: desde el registro hasta la conversión en solicitud de compra. El panel de administrador consume esos datos y muestra KPIs en tiempo real. Esta solución no requiere configuración nativa adicional ni costo extra, y se puede complementar con Firebase Analytics oficial cuando el proyecto escale a publicación en tiendas.

VIII

Entregables de Diseño y UI Kit

ATELIER · OutfitCatalog · Entrega Final · Dispositivos Móviles

1. Introducción

Este documento describe el sistema de diseño de ATELIER: la identidad visual, la paleta de colores, los tokens de tipografía y espaciado, los componentes de interfaz, y las pantallas principales. Está pensado como guía de referencia para mantener la consistencia visual en futuras actualizaciones del proyecto.

2. Identidad de Marca

El nombre ATELIER viene del francés y se refiere al espacio de trabajo de un artista o diseñador de moda. Elegimos este nombre porque transmite la idea de un espacio curado, donde las prendas no se venden al azar sino que forman parte de una propuesta visual.

Elemento	Definición
Nombre de marca	ATELIER
Nombre técnico del proyecto	OutfitCatalog
Categoría	Catálogo de moda, looks e inventario
Personalidad de marca	Sofisticada, visual, comercial, accesible
Tono visual	Premium, editorial, oscuro, elegante

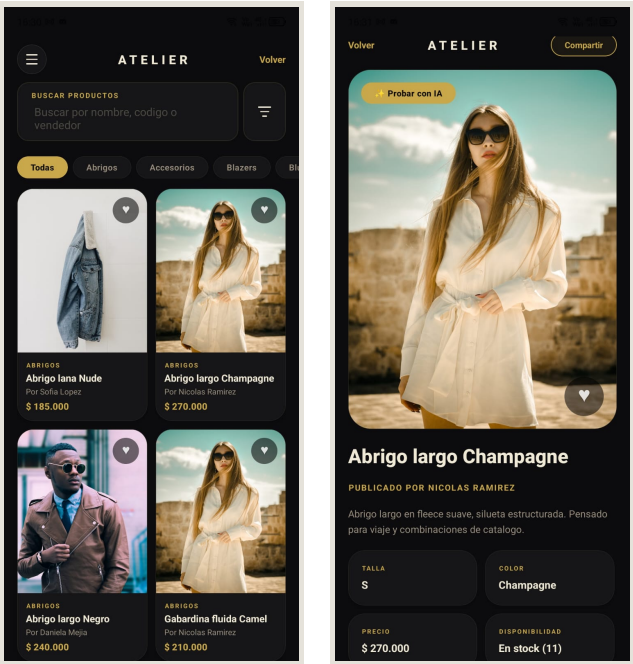
El diseño oscuro no fue caprichoso — en apps de moda y fotografía, el fondo oscuro hace que las imágenes de ropa resalten más. Instagram, ASOS y otras apps de moda usan esta lógica en sus modos oscuros. Para ATELIER, el modo oscuro es el modo principal.

3. Paleta de Colores

La paleta combina tonos muy oscuros para fondos, un dorado cálido como color de acción, y texto marfil para contraste. Los colores de estado (error, éxito) son saturados para ser visibles sobre el fondo oscuro.

Token	Color	Hex	Uso principal
Background	Negro profundo	#0C0C0E	Fondo de todas las pantallas
Surface	Gris muy oscuro	#161618	Cards, formularios, modales
Surface High	Gris oscuro elevado	#1F1F22	Dropdowns, superficies flotantes
Primary	Dorado cálido	#C9A84C	Botones principales, favoritos, accents
Primary Dark	Dorado presionado	#A8883A	Estado pressed del botón primario
Text Primary	Marfil	#F0EAD6	Títulos y texto principal

Token	Color	Hex	Uso principal
Text Secondary	Arena	#9B9080	Texto de apoyo, subtítulos
Text Muted	Gris apagado	#4E4A44	Placeholders, texto desactivado
Border	Gris borde	#2A2820	Bordes sutiles entre elementos
Border Light	Gris borde activo	#3A3630	Bordes de inputs activos
Error	Rojo	#E05252	Mensajes de error, alertas
Success	Verde	#52A882	Confirmaciones, stock disponible



Catálogo — fondo #0C0C0E, cards surface #161618, precio y accent en dorado #C9A84C, texto marfil #F0EAD6

Detalle de prenda — paleta completa en contexto: fondos, tipografía, botón primario dorado, badges de estado

4. Tipografía

Usamos la tipografía del sistema operativo (San Francisco en iOS, Roboto en Android) porque es la que el usuario ya conoce y espera. En lugar de importar una fuente externa, definimos jerarquías tipográficas claras con tamaños y pesos que definen la importancia visual de cada elemento.

Estilo	Tamaño	Peso	Uso
Brand	22 pt	800 ExtraBold	Logotipo textual ATELIER
Display	32 pt	800 ExtraBold	Títulos de pantalla grandes
Title	22 pt	700 Bold	Encabezados de sección
Label	11 pt	700 Bold	Etiquetas de formulario (mayúsculas)
Body	15 pt	400 Regular	Texto descriptivo de prendas

Estilo	Tamaño	Peso	Uso
Caption	12 pt	400 Regular	Metadatos, precio secundario, ayudas
Price	18 pt	700 Bold	Precio de prenda (destacado)

5. Espaciado y Radios

Usar un sistema de espaciado consistente hace que la interfaz se vea ordenada. Definimos los valores como tokens en theme.ts:

5.1 Espaciado

Token	Valor	Uso típico
xs	6 px	Espacio mínimo entre ícono y texto
sm	10 px	Padding interno de chips y badges
md	16 px	Padding estándar de cards y formularios
lg	24 px	Separación entre secciones
xl	36 px	Margen de pantalla lateral
xxl	52 px	Espacio antes de títulos principales

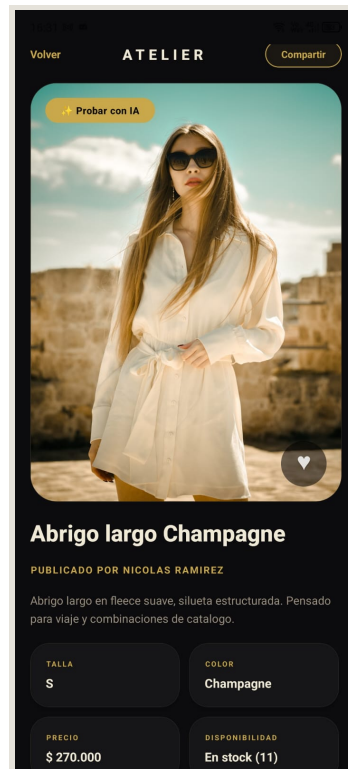
5.2 Radios de borde

Token	Valor	Uso típico
xs	4 px	Inputs pequeños
sm	10 px	Chips, badges
md	14 px	Cards de prenda
lg	20 px	Cards de look
xl	28 px	Botones principales
round	999 px	Avatares, botones circulares

6. Componentes Principales

6.1 Botón Primario

El botón principal usa el dorado #C9A84C como fondo con texto oscuro, radio xl (28px) y peso tipográfico Bold. Se usa para acciones irreversibles o de avance: "Crear cuenta", "Iniciar sesión", "Guardar look", "Enviar solicitud".

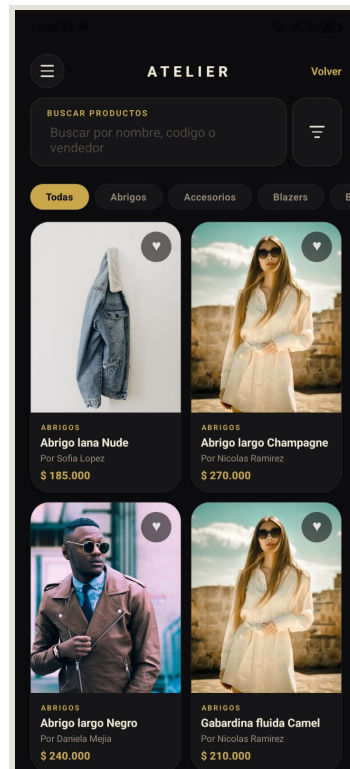


*Botón CTA en detalle de prenda
— radio xl (28px), fondo dorado
#C9A84C, texto oscuro, área
táctil ≥ 44pt*

6.2 Card de Prenda

La card de prenda es el componente más repetido en la app. Tiene:

- > Imagen a pantalla completa (aspectRatio 3:4)
- > Badge de categoría en la esquina superior
- > Nombre de la prenda
- > Precio en dorado
- > Ícono de favorito en la esquina



Grid de cards — imagen 3:4,
badge de categoría (Abrigos),
ícono corazón, precio en dorado,
fondo negro

6.3 Card de Look

La card de look muestra un collage de las prendas incluidas, el nombre del look y la cantidad de prendas. La portada del collage usa la primera imagen de las prendas del look.

6.4 Banner Offline

Cuando el usuario pierde conexión, aparece una barra en la parte superior de color superficie con ícono de red y texto:

Sin conexión • mostrando datos guardados localmente

El banner desaparece automáticamente cuando se recupera la red.

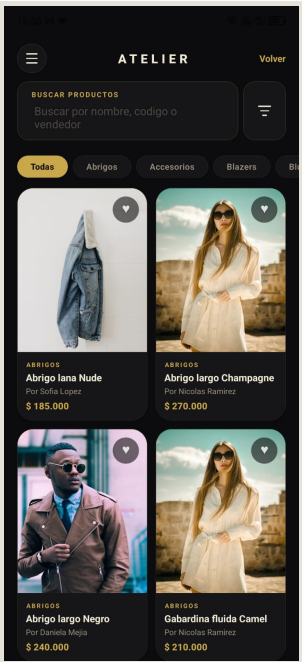
6.5 Estado Vacío

Cuando una pantalla no tiene datos (sin favoritos, sin looks, sin solicitudes), mostramos un estado vacío con ícono ilustrativo y un mensaje que explica qué hacer. Esto evita que el usuario se quede con una pantalla en blanco sin saber si es un error o es intencional.

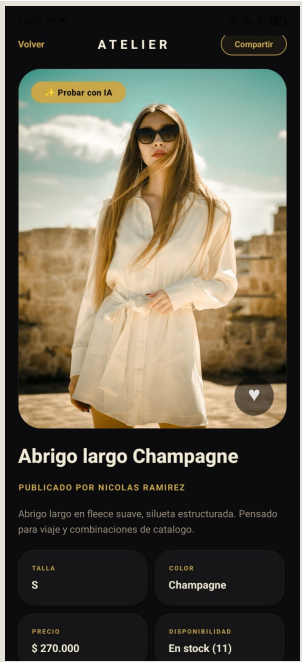
7. Pantallas Clave

Pantalla	Propósito	Rol que la usa
Onboarding	Presenta la app al primer ingreso	Todos
Login	Inicio de sesión	Todos

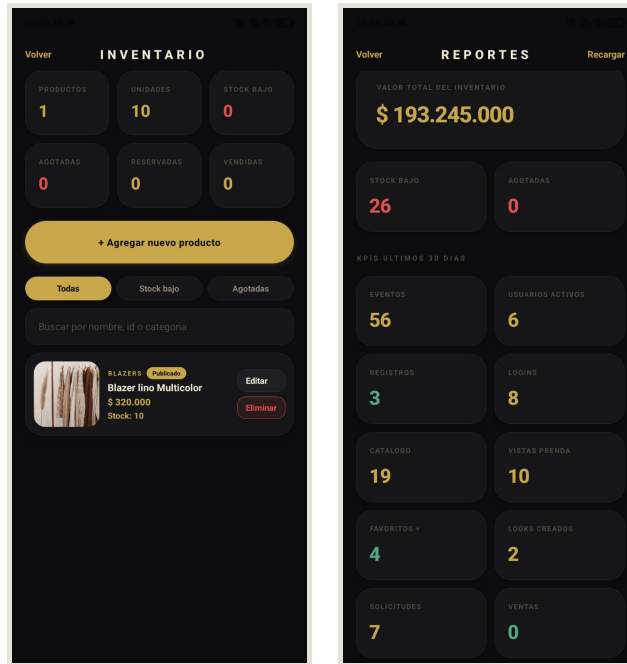
Pantalla	Propósito	Rol que la usa
Register	Crear cuenta con rol	Todos
GoogleRoleSelect	Seleccionar rol y teléfono (Google Sign-In nuevo)	Todos
UserHome	Inicio del cliente con acceso a catálogo	Cliente
VendorHome	Inicio del vendedor con métricas de inventario	Vendedor
AdminHome	Inicio del administrador con reportes	Administrador
GarmentGallery	Catálogo de prendas	Cliente
GarmentDetail	Detalle de prenda con acciones	Cliente
Favorites	Prendas guardadas y acceso a crear look	Cliente
Looks	Lista de looks creados	Cliente
LookDetail	Detalle de look con compartir y solicitar	Cliente
InventoryManagement	Gestión de prendas propias	Vendedor
PurchaseRequests	Solicitudes de compra (comprador/vendedor)	Cliente / Vendedor
AdminReports	Panel de KPIs y métricas	Administrador



Catálogo — pantalla principal del cliente



Detalle de prenda — ficha completa con acciones



*Inventario del vendedor —
gestión de prendas y stock*

*Panel de administrador —
KPIs de comportamiento en
tiempo real*

8. Archivos Fuente de Diseño

El repositorio incluye los assets de la aplicación en la carpeta /assets:

Archivo	Contenido
assets/icon.png	Ícono de la app (1024×1024)
assets/splash.png	Pantalla de carga (splash screen)
assets/adaptive-icon.png	Ícono adaptativo para Android
assets/favicon.png	Favicon para versión web

Los tokens de diseño (colores, tipografía, espaciado) están definidos en código en `src/theme.ts`. Esto significa que el sistema de diseño vive en el código fuente — cualquier cambio de color o espaciado se hace en un solo archivo y se aplica a toda la app.

Recomendación para versiones futuras: crear un archivo Figma con la paleta, componentes y flujos de pantalla. Esto facilitaría el trabajo de un diseñador externo o la iteración de diseño sin necesidad de correr la app para ver cambios visuales.

9. Guía de Accesibilidad Visual

Estas son las reglas de diseño que seguimos para garantizar que la interfaz sea legible para la mayor cantidad de usuarios posible:

- > El contraste entre texto primario (#F0EAD6) y fondo (#0C0C0E) supera el mínimo WCAG AA.
- > Los estados de error no dependen únicamente del color — acompañamos el color rojo con un mensaje de texto explicativo.

- > Los botones principales tienen área tocable de al menos 44×44 puntos para evitar errores de toque.
- > Los estados vacíos tienen texto descriptivo, no solo ilustraciones.

Lo que reconocemos como pendiente: agregar `accessibilityLabel` a todos los botones que usan solo íconos, y realizar una prueba formal con TalkBack activado en Android.

10. Conclusión

El sistema de diseño de ATELIER es coherente, bien definido y fácil de mantener. Los tokens en `theme.ts` permiten hacer cambios globales de estilo en minutos, y los componentes reutilizables garantizan consistencia visual en todas las pantallas. El paso natural para la siguiente versión es documentar el sistema en Figma para facilitar la colaboración con diseñadores y nuevos desarrolladores.

VIII

Plan de Despliegue y Mantenimiento

ATELIER · OutfitCatalog · Entrega Final · Dispositivos Móviles

1. Introducción

Este documento describe cómo está configurado el despliegue actual de ATELIER, qué se requiere para publicarlo en las tiendas oficiales, y cuál sería el plan de mantenimiento y soporte post-lanzamiento. También incluimos las credenciales de acceso necesarias para continuar el desarrollo o transferir el proyecto.

2. Estado Actual del Despliegue

ATELIER 1.0.0 está desplegado como APK para Android. El build fue generado en la nube mediante EAS Build, sin necesidad de tener Android Studio ni un Mac configurado localmente.

Elemento	Detalle
Plataforma	Android
Tipo de artefacto	APK
EAS Build ID	b56095a2-87a4-4dac-ae47-7f5d8599dee4
URL del APK	https://expo.dev/artifacts/eas/hFhAPFSHte8uvkEKgF2FTW.apk
Commit que generó el build	5bd8d25
Estado del build	FINISHED
Instalado en	7 dispositivos Android (prueba piloto)

3. Entregables Técnicos Incluidos

Entregable	Estado	Dónde está
Código fuente completo	Disponible	Repositorio Git
APK Android instalable	Generado	URL de EAS Build
Pruebas automatizadas	Incluidas	Carpeta <code>test/</code> y <code>src/core/services/*.test.ts</code>
Script de carga masiva	Incluido	<code>scripts/seed-massive.js</code>
Script de creación de admin	Incluido	<code>scripts/seed-admin.js</code>
Reglas de seguridad Firestore	Incluidas	Documento 09-reglas-seguridad-firestore.md
Configuración EAS	Incluida	<code>eas.json</code>
Variables de entorno de referencia	Documentadas	Sección 5 de este documento

4. Cómo Ejecutar el Proyecto Localmente

4.1 Requisitos

- > Node.js LTS (recomendado 20+)
- > npm
- > Cuenta en Expo (expo.dev) para builds en la nube
- > Proyecto Firebase con Auth y Firestore habilitados
- > Cuenta Cloudinary para almacenamiento de imágenes
- > Android Studio o un dispositivo físico Android para probar

4.2 Pasos de instalación

```
BASH

# Clonar el repositorio
git clone https://github.com/andresbot/OutfitCatalog.git
cd OutfitCatalog

# Instalar dependencias
npm install

# Verificar que TypeScript compila sin errores
npm run build

# Ejecutar pruebas
npm test

# Iniciar en modo desarrollo (requiere Expo Go o emulador)
npm run start
```

4.3 Generar un nuevo APK

```
BASH

npx eas-cli@latest build -p android --profile preview --non-interactive
```

El APK queda disponible en el panel de EAS (expo.dev/accounts/.../builds) en aproximadamente 10–20 minutos.

5. Variables de Entorno Requeridas

El proyecto usa variables de entorno con prefijo EXPOPUBLIC que EAS inyecta en el bundle. Para desarrollo local, crear un archivo .env.local con:

```
EXPO_PUBLIC_FIREBASE_API_KEY=
EXPO_PUBLIC_FIREBASE_AUTH_DOMAIN=
EXPO_PUBLIC_FIREBASE_PROJECT_ID=
EXPO_PUBLIC_FIREBASE_STORAGE_BUCKET=
EXPO_PUBLIC_FIREBASE_MESSAGING_SENDER_ID=
EXPO_PUBLIC_FIREBASE_APP_ID=
EXPO_PUBLIC_GOOGLE_ANDROID_CLIENT_ID=
EXPO_PUBLIC_GOOGLE_WEB_CLIENT_ID=
EXPO_PUBLIC_CLOUDINARY_CLOUD_NAME=
EXPO_PUBLIC_CLOUDINARY_UPLOAD=
```

Estos valores están configurados en EAS Secrets para el proyecto de producción. Para transferir el proyecto a otra cuenta, estos valores deben ser actualizados en el panel de EAS.

6. Acceso a las Credenciales del Proyecto

6.1 Firebase

Plataforma	Acceso
Firebase Console	console.firebase.google.com
Proyecto	outfitcatalog (o el nombre configurado)
Servicios activos	Authentication, Firestore, (Cloudinary externo)
Cuenta administradora	La cuenta Google con la que se creó el proyecto

Para revisar la analítica en Firestore: ir a Firestore → colección analyticsEvents.

Para revisar usuarios registrados: Firebase → Authentication → Users.

6.2 EAS Build (Expo)

Plataforma	Acceso
Panel EAS	expo.dev
Cuenta	La cuenta Expo de los autores
Proyecto	OutfitCatalog

Los builds históricos y el APK de esta entrega están disponibles en ese panel.

6.3 Google Play Console (pendiente)

La app aún no está publicada. Para publicarla, se requiere:

- > Crear una cuenta en Google Play Console (\$25 USD, pago único)
- > Generar un build AAB (Android App Bundle) en lugar de APK:

BASH

```
npx eas-cli@latest build -p android --profile production
```

- > Subir el AAB, completar la ficha de tienda y pasar la revisión de Google

6.4 Apple Developer (pendiente)

La app no tiene build iOS. Para publicarla en App Store:

- > Cuenta Apple Developer (\$99 USD/año)
- > Build iOS con EAS:

BASH

```
npx eas-cli@latest build -p ios --profile production
```

- > TestFlight para pruebas internas
- > Revisión de Apple (7-14 días)

7. Creación del Usuario Administrador

El rol de administrador no se puede crear desde la app por seguridad. Se crea con un script que usa una cuenta de servicio de Firebase:

```
BASH
npm run seed:admin -- --service-account "C:\ruta\service-account.json"
```

Credenciales del administrador de prueba generadas por el seed:

Campo	Valor
Email	admin@outfit.test
Contraseña	Admin123!
Rol	admin

8. Carga Masiva de Datos de Prueba

Para poblar la app con datos de demo antes de una presentación:

```
BASH
npm run seed:massive -- \
  --service-account "ruta/service-account.json" \
  --auth-users \
  --vendors 40 \
  --clients 250 \
  --admins 3 \
  --garments 100
```

Esto crea 393 documentos en Firestore: 293 usuarios y 100 prendas distribuidas entre los vendedores. Los vendedores tienen teléfono +573104221496 por defecto para que el flujo de WhatsApp funcione en demostraciones.

Importante: hacer un respaldo de Firestore antes de ejecutar el seed en un proyecto con datos reales.

9. Plan de Soporte y Mantenimiento

9.1 Prioridades de respuesta

Nivel	Ejemplo	Tiempo de atención sugerido
Crítico	La app no abre, login bloqueado, pérdida de datos	24 horas
Alto	Solicitudes no se crean, inventario no actualiza, catálogo vacío	48 horas
Medio	Error visual, texto incorrecto, lentitud puntual	3–5 días hábiles
Bajo	Mejora estética, funcionalidad secundaria	Próxima versión

9.2 Mantenimiento preventivo

Antes de cada release nuevo:

1. Ejecutar npm run build y verificar que TypeScript no tiene errores.
2. Ejecutar npm test y verificar que todos los tests pasan.
3. Probar el flujo principal en un dispositivo Android físico: registro → catálogo → favorito → solicitud.
4. Revisar que las reglas de Firestore no rompan ningún flujo después de cambios.
5. Hacer respaldo de Firestore si se van a ejecutar migraciones de datos.

9.3 Monitoreo en producción

Herramientas que recomendamos integrar cuando la app llegue a tiendas:

Herramienta	Para qué	Costo
Firebase Crashlytics	Crashes en producción con stack trace automático	Gratuito
Firebase Analytics	Retención, embudos, audiencias en GA4	Gratuito
Firebase Performance	Tiempos de carga en dispositivos reales	Gratuito
Google Play Console	Reviews, crashes reportados, ANRs	Incluido en cuenta

10. Control de Versiones y Ramas

El repositorio usa Git con rama principal main. Recomendamos este flujo para el mantenimiento futuro:

- > main — versiones estables y publicadas
- > develop — integración de funcionalidades en desarrollo
- > feature/nombre — ramas por funcionalidad
- > Tags: v1.0.0, v1.1.0, etc. para cada versión publicada

El commit que corresponde a esta entrega es 5bd8d25.

11. Riesgos Post-Entrega

Riesgo	Probabilidad	Impacto	Mitigación
Cambio de reglas Firebase bloquea un flujo	Media	Alto	Versionar reglas, probar después de cada cambio
Expiración de API keys	Baja	Alto	Documentar rotación de variables de entorno
Crecimiento del catálogo afecta rendimiento	Media	Medio	Agregar paginación en catálogo
Bug crítico post-lanzamiento	Media	Alto	Integrar Crashlytics antes de publicar
Publicación iOS sin cuenta Apple	Alta	Medio	Planificar cuenta de desarrollador para siguiente versión

12. Conclusión

El despliegue Android está resuelto y verificado. El APK generado con EAS Build funciona en dispositivos reales, el código está en un repositorio Git con historial limpio, y las credenciales están documentadas para facilitar la transferencia o continuación del proyecto. Los próximos pasos naturales son integrar Crashlytics, agregar paginación al catálogo, y preparar el build para publicación en Google Play Store.